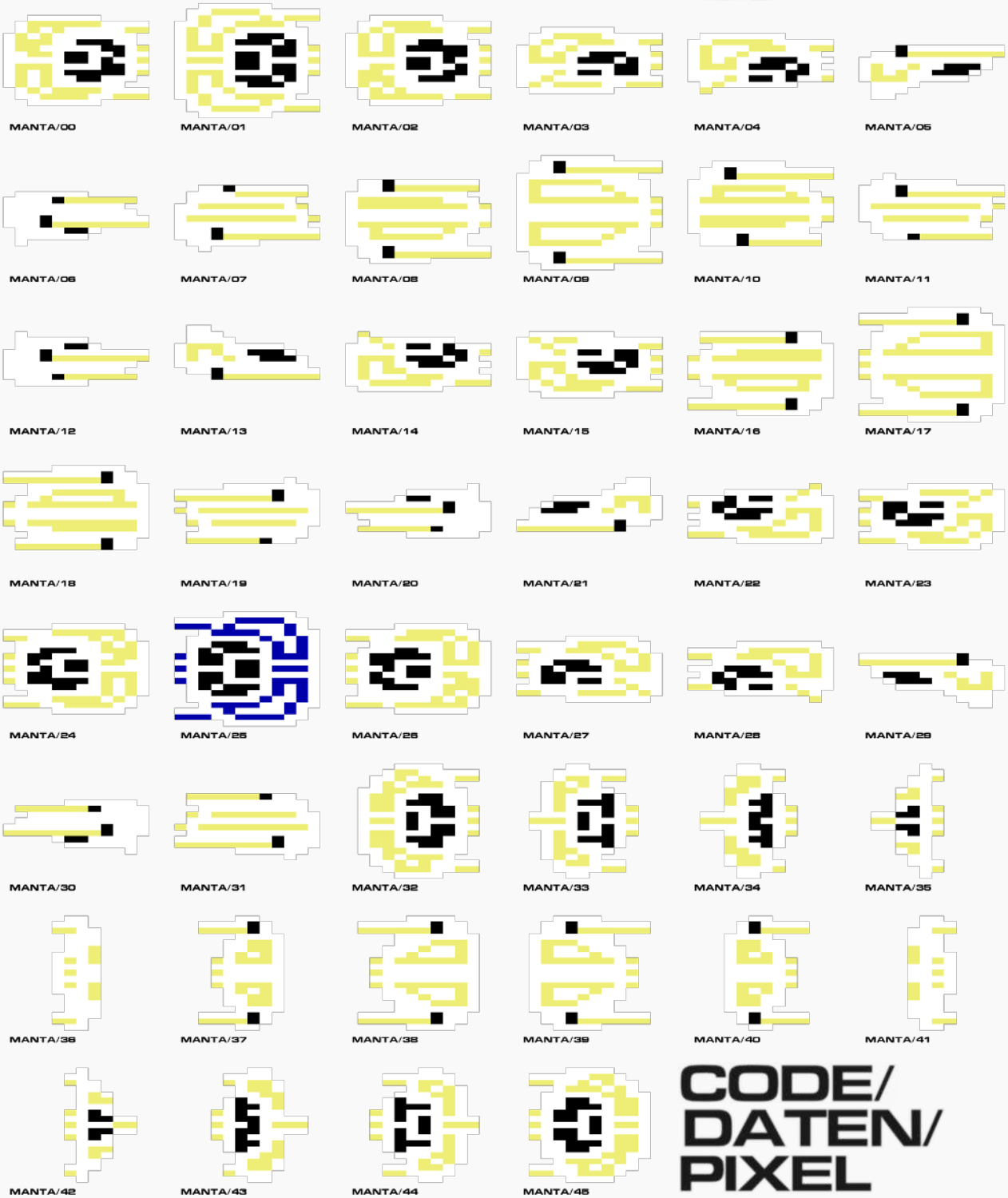
URIDIUM/ FRAKTION



CODE/ DATEN/ PIXEL

URIDIUM/ FRAKTION

For Edna.

© Rob Hogan 2026, All Rights Reserved.
Edition Date: Saturday 28th March, 2026

This work is licensed under a Creative Commons "Attribution-NonCommercial-ShareAlike 3.0 Unported" license.

Contents

MANTA/SPIN 7

DROP/SHIP 15

ABOUT/THIS/BOOK

This book isn't about the history of *Tempest* or its importance in the history of computer games. This is a book about computer code. Each chapter aims to be an easily digested sketch of how a specific feature or effect was achieved. In the case of *Tempest* this means sifting through the 6502 assembly code David Theurer wrote by hand(!) to create the game, and wrangling with the bits and bytes of Atari's Quadrascan Vector Graphics. For *Tempest 2000* it means familiarizing ourselves with Motorola 68K assembly code and the Jaguar's powerful hardware platform.

There are lots of fascinating bits and pieces to pore over. And it is interesting to see how the years that separate the two games leave some things unchanged and many other things changed utterly. I've put together this book of squibs in the hope they will allow you, the reader, to digest small bite-sized insights into each game. And I hope, by interleaving elements from *Tempest* and *Tempest 2000*, to enable you to compare the parallel lives in the design and execution of separate incarnations of the same classic video game.

Rob Hogan (@mwenge)
2026

URIDIUM/
FRAKTION

MANTA/SPIN

When waiting to start the first level, the player's Manta ship spins elegantly in the center of the screen. This simple animation uses an enormous number of sprites - 16 in total.

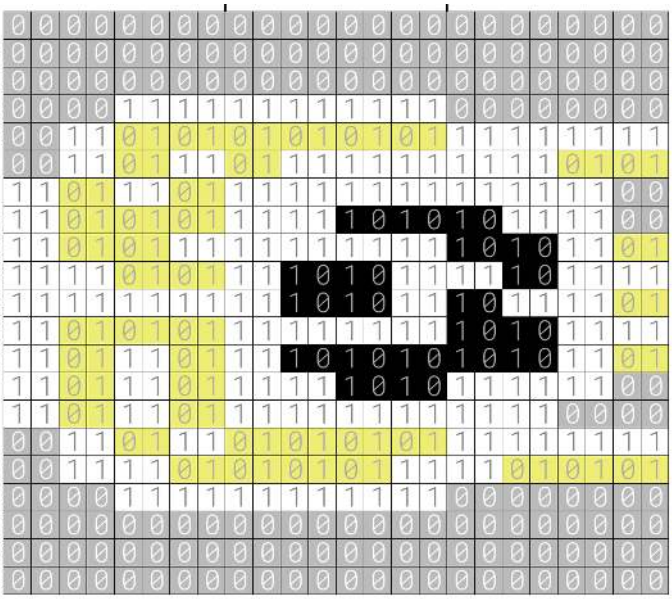
The first phase of the spin - - does not start from where we might expect. The rotation of the initial ship is negative so that we start from a slightly rotated position.

The next frame in the animation - - rotates the ship to the opposite roll position.

So the Manta starts tilted away from us, rotates to a flat position, and then begins its roll towards us. Notice the subtly different assymetry in the cockpit of the Manta when it is rolled away from us compared to when it is rolled towards us. It is easiest to compare this difference in detail on the following page where we blow up each of the tilted sprites to see how they are constructed from their source data.

URIDIUM/MANTA/00

SPEC:MANTA
AZIMUTH:090°
OFFSET:$40
ELEVATION:000°
RAM:$5000
ROLL:015°

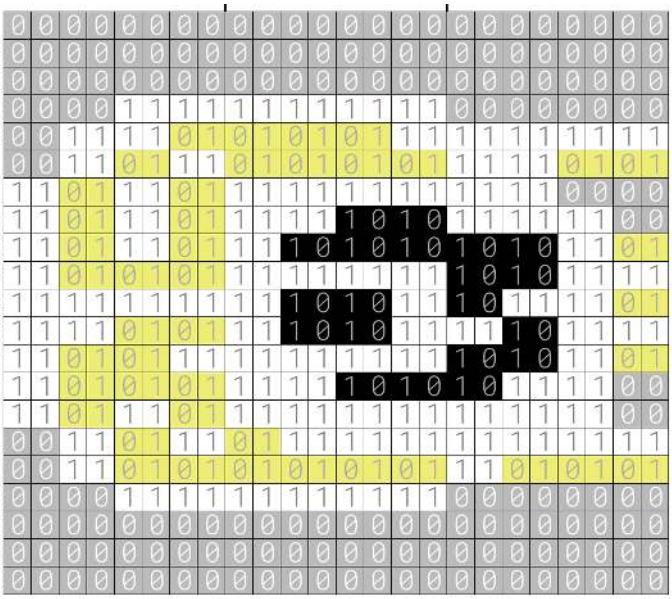


```
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $0F,$FF,$00
.BYTE $35,$55,$FF
.BYTE $37,$7F,$F5
.BYTE $DD,$FF,$FC
.BYTE $D5,$FA,$BC
.BYTE $D7,$FF,$AD
.BYTE $F5,$EB,$EF
.BYTE $FF,$EB,$BD
.BYTE $D5,$FF,$AF
.BYTE $DD,$EA,$AD
.BYTE $DD,$FA,$FC
.BYTE $DD,$FF,$F0
.BYTE $37,$55,$FF
.BYTE $3D,$57,$D5
.BYTE $0F,$FF,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
```

BASE_COLOR:□ MULTI_COLOR0:□ MULTI_COLOR1:■

URIDIUM/MANTA/02

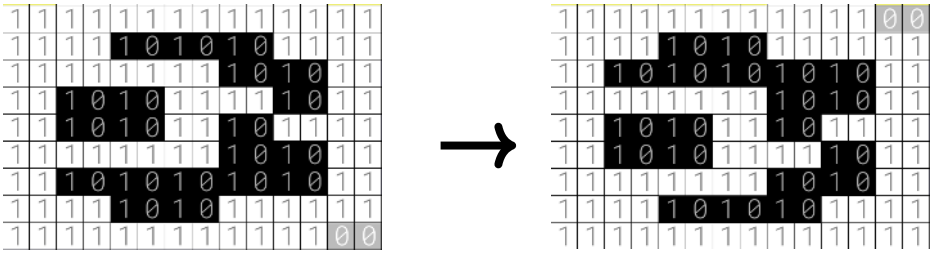
SPEC:MANTA_2
AZIMUTH:090°
OFFSET:$42
ELEVATION:000°
RAM:$5080
ROLL:015°



```
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $0F,$FF,$00
.BYTE $3D,$57,$FF
.BYTE $37,$55,$F5
.BYTE $DD,$FF,$F0
.BYTE $DD,$FA,$FC
.BYTE $DD,$EA,$AD
.BYTE $D5,$FF,$AF
.BYTE $FF,$EB,$BD
.BYTE $F5,$EB,$EF
.BYTE $D7,$FF,$AD
.BYTE $D5,$FA,$BC
.BYTE $DD,$FF,$FC
.BYTE $37,$7F,$FF
.BYTE $35,$55,$D5
.BYTE $0F,$FF,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
```

BASE_COLOR:□ MULTI_COLOR0:□ MULTI_COLOR1:■

Closer inspection reveals a simple trick that behind this detailing: the yellow and black regions have simply had their order reversed and are mirror images of each other. For example if we take a look at the region representing the cockpit:

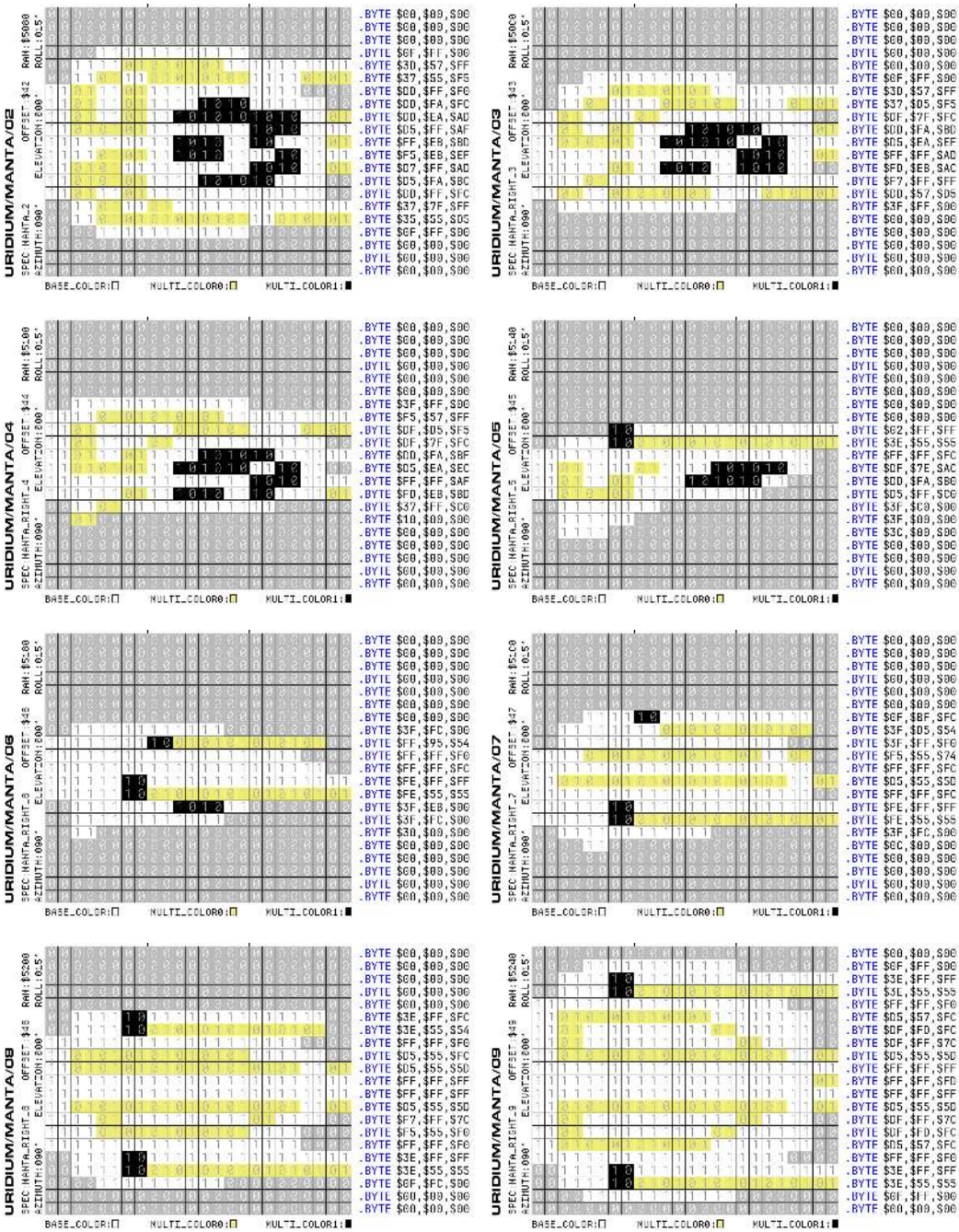


In its simplest terms the vertical order of the black lines detailing the cockpit have been reversed and this alone is enough to suggest the difference in attitude along the Y co-ordinate.

Examining this detail may also give you a sense of how the sprites themselves are defined. In the diagram above, for example, it may now be obvious that the black colour is defined by a bit pair of `10` and that the white colour is defined by a bit pair of `11`. In the diagrams opposite, we can further see that yellow is defined by the bit paid `01` and the transparent background of the sprite by `00`.

In the diagrams opposite and elsewhere we have given the three bytes that together make up the 12 bit pairs constituting each of the 21 lines in the sprite, a total of 63 bytes in total. This is the source data used by *Uridium's* 6502 assembler code to build each sprite using the straightforward process of mapping each bit pair to a corresponding pair of coloured pixels. An important feature of this sprite data is that it is stored contiguously for all sprites in the computer's RAM, or memory area, starting at address `$5000`. This is important because the order in which the sprite data is stored corresponds exactly to the order of each frame in the manta spin animation.

We can get a better sense of this by observing the RAM address of each sprite in the section of animation sequence between the ship faced towards and faced away. The first sprite in the sequence is at address `$5040`, the second at `$5080`, and so on. At each step, to get the next frame in the animation all that has to be done is to increment the position in RAM we are adressing by `$40`.


URIDIUM/MANTA/02
SPEC MANTA_2 OFFSET $42 RAH:$5080
AZIMUTH:090° ELEVATION:200° ROLL:0.5°
BASE_COLOR: MULTI_COLOR0: MULTI_COLOR1:
.BYTE $00,$09,$90
.BYTE $00,$09,$90
.BYTE $00,$09,$90
.BYIL $0F,$FF,$90
.BYTE $3D,$57,$FF
.BYTE $37,$55,$F5
.BYTE $DD,$FF,$F0
.BYTE $DD,$FA,$FC
.BYTE $DD,$EA,$AD
.BYTE $D5,$FF,$AF
.BYTE $FF,$EB,$BD
.BYTE $F5,$EB,$EF
.BYTE $D7,$FF,$AD
.BYTE $D5,$FA,$BC
.BYIE $DD,$FF,$FC
.BYTE $37,$7F,$FF
.BYTE $35,$55,$D5
.BYTE $0F,$FF,$90
.BYTE $00,$09,$90
.BYIL $00,$09,$90
.BYTE $00,$09,$90
URIDIUM/MANTA/03
SPEC MANTA_RIGHT_3 OFFSET $43 RAH:$50C0
AZIMUTH:090° ELEVATION:200° ROLL:0.5°
BASE_COLOR: MULTI_COLOR0: MULTI_COLOR1:
.BYTE $00,$09,$90
.BYTE $00,$09,$90
.BYTE $00,$09,$90
.BYIL $00,$09,$90
.BYTE $00,$09,$90
.BYTE $0F,$FF,$90
.BYTE $3D,$57,$FF
.BYTE $37,$D5,$F5
.BYIE $DF,$7F,$FC
.BYTE $DD,$FA,$BD
.BYTE $D5,$FA,$FF
.BYTE $FF,$FF,$AD
.BYTE $FD,$EB,$AC
.BYTE $F7,$FF,$FF
.BYIE $DD,$57,$D5
.BYTE $3F,$FF,$90
.BYTE $00,$09,$90
.BYTE $00,$09,$90
.BYTE $00,$09,$90
.BYIL $00,$09,$90
.BYTE $00,$09,$90
URIDIUM/MANTA/04
SPEC MANTA_RIGHT_4 OFFSET $44 RAH:$5100
AZIMUTH:090° ELEVATION:200° ROLL:0.5°
BASE_COLOR: MULTI_COLOR0: MULTI_COLOR1:
.BYTE $00,$09,$90
.BYTE $00,$09,$90
.BYTE $00,$09,$90
.BYIL $00,$09,$90
.BYTE $00,$09,$90
.BYTE $00,$09,$90
.BYTE $3F,$FF,$90
.BYTE $F5,$57,$FF
.BYTE $DF,$D5,$F5
.BYTE $DF,$7F,$FC
.BYTE $DD,$FA,$BF
.BYTE $D5,$EA,$EC
.BYTE $FF,$FF,$AF
.BYTE $FD,$EB,$BD
.BYIE $37,$FF,$C0
.BYTE $10,$09,$90
.BYTE $00,$09,$90
.BYTE $00,$09,$90
.BYTE $00,$09,$90
.BYIL $00,$09,$90
.BYTE $00,$09,$90
URIDIUM/MANTA/05
SPEC MANTA_RIGHT_5 OFFSET $45 RAH:$5140
AZIMUTH:090° ELEVATION:200° ROLL:0.5°
BASE_COLOR: MULTI_COLOR0: MULTI_COLOR1:
.BYTE $00,$09,$90
.BYTE $00,$09,$90
.BYTE $00,$09,$90
.BYIL $00,$09,$90
.BYTE $00,$09,$90
.BYTE $00,$09,$90
.BYTE $00,$09,$90
.BYTE $00,$09,$90
.BYIE $C2,$FF,$FF
.BYTE $3E,$55,$55
.BYTE $FF,$FF,$FC
.BYTE $DF,$7E,$AC
.BYTE $DD,$FA,$B0
.BYTE $D5,$FF,$C0
.BYTE $3F,$C0,$90
.BYTE $3F,$09,$90
.BYTE $3C,$09,$90
.BYTE $00,$09,$90
.BYTE $00,$09,$90
.BYIL $00,$09,$90
.BYTE $00,$09,$90
URIDIUM/MANTA/06
SPEC MANTA_RIGHT_6 OFFSET $46 RAH:$5180
AZIMUTH:090° ELEVATION:200° ROLL:0.5°
BASE_COLOR: MULTI_COLOR0: MULTI_COLOR1:
.BYTE $00,$09,$90
.BYTE $00,$09,$90
.BYTE $00,$09,$90
.BYIL $00,$09,$90
.BYTE $00,$09,$90
.BYTE $00,$09,$90
.BYTE $00,$09,$90
.BYTE $3F,$FC,$90
.BYIE $FF,$95,$54
.BYIE $FF,$FF,$F0
.BYTE $FF,$FF,$FC
.BYTE $FE,$FF,$FF
.BYTE $FE,$55,$55
.BYTE $3F,$EB,$90
.BYIE $3F,$FC,$90
.BYTE $30,$09,$90
.BYTE $00,$09,$90
.BYTE $00,$09,$90
.BYTE $00,$09,$90
.BYIL $00,$09,$90
.BYTE $00,$09,$90
URIDIUM/MANTA/07
SPEC MANTA_RIGHT_7 OFFSET $47 RAH:$51C0
AZIMUTH:090° ELEVATION:200° ROLL:0.5°
BASE_COLOR: MULTI_COLOR0: MULTI_COLOR1:
.BYTE $00,$09,$90
.BYTE $00,$09,$90
.BYTE $00,$09,$90
.BYIL $00,$09,$90
.BYTE $00,$09,$90
.BYTE $00,$09,$90
.BYTE $00,$09,$90
.BYTE $0F,$BF,$FC
.BYTE $3F,$D5,$54
.BYIE $3F,$FF,$F0
.BYIE $F5,$55,$74
.BYTE $FF,$FF,$FC
.BYTE $D5,$55,$50
.BYTE $FF,$FF,$FC
.BYTE $FE,$FF,$FF
.BYIE $FE,$55,$55
.BYTE $3F,$FC,$90
.BYTE $0C,$09,$90
.BYTE $00,$09,$90
.BYTE $00,$09,$90
.BYTE $00,$09,$90
.BYIL $00,$09,$90
.BYTE $00,$09,$90
URIDIUM/MANTA/08
SPEC MANTA_RIGHT_8 OFFSET $48 RAH:$5200
AZIMUTH:090° ELEVATION:200° ROLL:0.5°
BASE_COLOR: MULTI_COLOR0: MULTI_COLOR1:
.BYTE $00,$09,$90
.BYTE $00,$09,$90
.BYTE $00,$09,$90
.BYIL $00,$09,$90
.BYTE $00,$09,$90
.BYTE $3E,$FF,$FC
.BYTE $3E,$55,$54
.BYTE $FF,$FF,$F0
.BYIE $D5,$55,$FC
.BYIE $D5,$55,$50
.BYTE $FF,$FF,$FF
.BYTE $FF,$FF,$FF
.BYTE $D5,$55,$50
.BYTE $F7,$FF,$7C
.BYIL $F5,$55,$F0
.BYTE $FF,$FF,$F0
.BYTE $3E,$FF,$FF
.BYTE $3E,$55,$55
.BYTE $0F,$FC,$90
.BYIL $00,$09,$90
.BYTE $00,$09,$90
URIDIUM/MANTA/09
SPEC MANTA_RIGHT_9 OFFSET $49 RAH:$5240
AZIMUTH:090° ELEVATION:200° ROLL:0.5°
BASE_COLOR: MULTI_COLOR0: MULTI_COLOR1:
.BYTE $00,$09,$90
.BYTE $0F,$FF,$90
.BYTE $3E,$FF,$FF
.BYIL $3L,$55,$55
.BYTE $FF,$FF,$F0
.BYTE $D5,$57,$FC
.BYTE $DF,$FD,$FC
.BYTE $DF,$FF,$7C
.BYIL $D5,$55,$5D
.BYTE $FF,$FF,$FF
.BYTE $FF,$FF,$FD
.BYTE $FF,$FF,$FF
.BYTE $D5,$55,$5D
.BYTE $DF,$FF,$7C
.BYIL $DF,$FD,$FC
.BYTE $D5,$57,$FC
.BYTE $FF,$FF,$F0
.BYTE $3E,$FF,$FF
.BYTE $3E,$55,$55
.BYIL $0L,$II,$90
.BYTE $00,$09,$90

This approach does not hold for the second half of the animation sequence, or at least not all of it. As we can see below, the final two frames of the sequence reset back to address $5000 to stitch in the start of the animation and complete the loop.

But this is not a wrinkle the animation needs to worry about. All it needs to do is loop across the sprites that constitute the full sequence. It is indifferent to the fact that the first frame in RAM is not a manta sitting flush to our point of view.

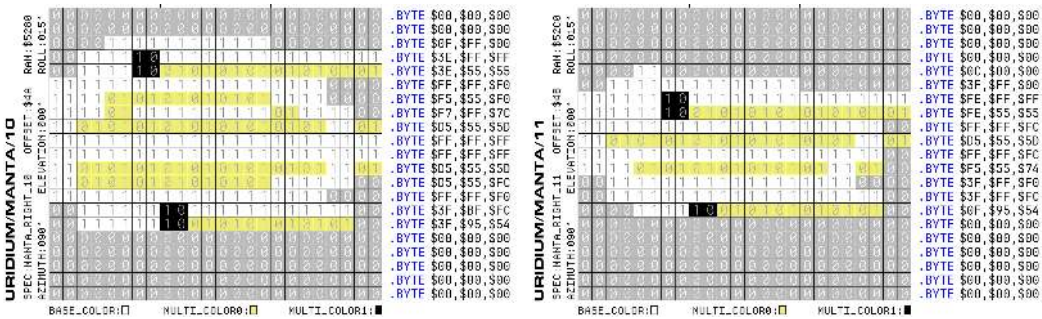
Let's take a look at how it is implemented in the routine responsible for animating the spinning manta. It's thankfully very compact and easy to digest. We would expect it to iterate over the sprites from `MANTA_0` to `MANTA_15`, which is exactly what it does - as we noted, all that matters to it is that the sprites are ordered contiguously in RAM. Selecting the next sprite for preparation (`GetCurrentSprite`) and display (`DisplaySprite`) is a simple matter of incrementing the index `currentSpriteValue` that points to the current sprite until we reach `MANTA_16`, at which point we reset our `currentSpriteValue` back to `MANTA_0`.

```
SpinningShipLoop
    JSR CheckInput            ; Increment frameCount and check input.
    LDA firePressed           ; Was 'Fire' pressed?
    BEQ AnimationOver         ; If so, stop animating ship.
    LDA frameCount            ; Is the sequence over?
    BEQ AnimationOver         ; If so, stop animating ship.
    JSR MaybeShowPauseScreen  ; Check if 'Pause' pressed.
    JSR GetCurrentSprite      ; Prepare current sprite.
    INC currentSpriteValue    ; Increment the sprite index.
    LDA currentSpriteValue    ; Store it in 'A' register.
    CMP #MANTA_16             ; Is it MANTA_16 yet?
    BCC Repeat                ; If not, display sprite and loop.
    LDA #MANTA_00             ; Otherwise, fetch MANTA_00 ..
    STA currentSpriteValue    ; .. and store it as current sprite.
Repeat
    JSR DisplayCurrentSprite  ; Display our selected sprite.
    JMP SpinningShipLoop      ; Loop back to top again.
```

But there is one complication we haven't yet accounted for: how does the animation ensure that when `frameCount` has reached zero, and the animation is over, that we finish with the Manta facing upwards: ? The answer lies in the initial value that we set for `frameCount`. This is $BF or 191 in decimal:

```
    LDA #$BF          ; Store 191 in the A register.
    STA frameCount    ; Store it as our frameCount.
```

MANTA/SPIN


URIDIUM/MANTA/10
SPEC MANTA_RIGHT_10 OFFSET $4A RAH:$5200
AZIMUTH:090° ELEVATION:200° ROLL:0.5°
BASE_COLOR:
MULTI_COLOR0:
MULTI_COLOR1:
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $0F,$FF,$00
.BYTE $3L,$FF,$FF
.BYTE $3E,$55,$55
.BYTE $FF,$FF,$F0
.BYTE $F5,$55,$F0
.BYTE $F7,$FF,$7C
.BYTE $D5,$55,$5D
.BYTE $FF,$FF,$FF
.BYTE $FF,$FF,$FF
.BYTE $D5,$55,$5D
.BYTE $D5,$55,$FC
.BYTE $FF,$FF,$F0
.BYTE $3F,$BF,$FC
.BYTE $3F,$95,$54
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
URIDIUM/MANTA/11
SPEC MANTA_RIGHT_11 OFFSET $4B RAH:$5200
AZIMUTH:090° ELEVATION:200° ROLL:0.5°
BASE_COLOR:
MULTI_COLOR0:
MULTI_COLOR1:
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $0C,$00,$00
.BYTE $3F,$FF,$00
.BYTE $FE,$FF,$FF
.BYTE $FE,$55,$55
.BYTE $FF,$FF,$FC
.BYTE $D5,$55,$5D
.BYTE $FF,$FF,$FC
.BYTE $F5,$55,$74
.BYTE $3F,$FF,$F0
.BYTE $3F,$FF,$FC
.BYTE $0F,$95,$54
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
URIDIUM/MANTA/12
SPEC MANTA_RIGHT_12 OFFSET $4C RAH:$5300
AZIMUTH:090° ELEVATION:200° ROLL:0.5°
BASE_COLOR:
MULTI_COLOR0:
MULTI_COLOR1:
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $30,$00,$00
.BYTE $3F,$FC,$00
.BYTE $3F,$EB,$00
.BYTE $FE,$FF,$FF
.BYTE $FE,$55,$55
.BYTE $FF,$FF,$FC
.BYTE $FF,$FF,$F0
.BYTE $FF,$95,$54
.BYTE $3F,$FC,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
URIDIUM/MANTA/13
SPEC MANTA_RIGHT_13 OFFSET $4D RAH:$5400
AZIMUTH:090° ELEVATION:200° ROLL:0.5°
BASE_COLOR:
MULTI_COLOR0:
MULTI_COLOR1:
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $3C,$00,$00
.BYTE $3F,$00,$00
.BYTE $3F,$C0,$00
.BYTE $D5,$FF,$C0
.BYTE $DD,$FA,$B0
.BYTE $DF,$7E,$AC
.BYTE $FF,$FF,$FC
.BYTE $3E,$FF,$FF
.BYTE $02,$55,$55
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
URIDIUM/MANTA/14
SPEC MANTA_RIGHT_14 OFFSET $4E RAH:$5300
AZIMUTH:090° ELEVATION:200° ROLL:0.5°
BASE_COLOR:
MULTI_COLOR0:
MULTI_COLOR1:
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $10,$00,$00
.BYTE $37,$FF,$C0
.BYTE $F0,$EB,$BF
.BYTE $FF,$FF,$AD
.BYTE $D5,$EA,$EC
.BYTE $DD,$FA,$BF
.BYTE $DD,$7F,$FC
.BYTE $DF,$D5,$FF
.BYTE $F5,$57,$D5
.BYTE $3F,$FF,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
URIDIUM/MANTA/15
SPEC MANTA_RIGHT_15 OFFSET $4F RAH:$5300
AZIMUTH:090° ELEVATION:200° ROLL:0.5°
BASE_COLOR:
MULTI_COLOR0:
MULTI_COLOR1:
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $3F,$FF,$00
.BYTE $DD,$7F,$FF
.BYTE $F7,$FF,$F5
.BYTE $FD,$EB,$AC
.BYTE $FF,$FF,$AD
.BYTE $D5,$FA,$BF
.BYTE $DD,$FA,$B0
.BYTE $DF,$7F,$FC
.BYTE $37,$55,$FF
.BYTE $3D,$57,$D5
.BYTE $0F,$FF,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
URIDIUM/MANTA/00
SPEC MANTA OFFSET $40 RAH:$5000
AZIMUTH:090° ELEVATION:200° ROLL:0.5°
BASE_COLOR:
MULTI_COLOR0:
MULTI_COLOR1:
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $0F,$FF,$00
.BYTE $35,$55,$FF
.BYTE $37,$7F,$F5
.BYTE $DD,$FF,$FC
.BYTE $D5,$FA,$BC
.BYTE $D7,$FF,$AD
.BYTE $F5,$EB,$EF
.BYTE $FF,$FB,$BD
.BYTE $D5,$FF,$AF
.BYTE $DD,$EA,$AD
.BYTE $DD,$FA,$FC
.BYTE $UD,$FF,$F0
.BYTE $37,$55,$FF
.BYTE $3D,$57,$D5
.BYTE $0F,$FF,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
.BYTE $00,$00,$00
URIDIUM/MANTA/01
SPEC MANTA1 OFFSET $41 RAH:$5040
AZIMUTH:090° ELEVATION:200° ROLL:0.5°
BASE_COLOR:
MULTI_COLOR0:
MULTI_COLOR1:
.BYTE $00,$00,$00
.BYTE $0F,$FF,$00
.BYTE $3D,$57,$FF
.BYTE $37,$FD,$D5
.BYTE $F7,$D5,$70
.BYTE $DF,$7F,$FC
.BYTE $DD,$FA,$BC
.BYTE $DD,$EA,$EC
.BYTE $D5,$FF,$AD
.BYTE $FF,$EB,$AF
.BYTE $D5,$FB,$FD
.BYTE $FF,$EB,$AF
.BYTE $D5,$FF,$AD
.BYTE $DD,$EA,$EC
.BYTE $UD,$FA,$BC
.BYTE $DF,$7F,$FC
.BYTE $F7,$D5,$70
.BYTE $37,$FD,$FF
.BYTE $3D,$57,$D5
.BYTE $0L,$FL,$00
.BYTE $00,$00,$00

A starting value of 191 for `frameCount` gives us 65 keyframes of animation in total until we reach 256 and roll around, like a clock, back to zero. With 16 frames for each complete spin of the Manta this allows us 4 complete spins (64 frames) with one left over before `frameCount` will roll over to zero. It is this left over frame that will push us past `MANTA_00` - - and finish at the desired spot of `MANTA_1` - .

The Manta does not only spin on its axis when travelling from left to right. The effect is also present in the right-to-left direction during the enemy ship's destruction sequence.

This effect, along with a number of other Manta maneouvres, is achieved with a more involved use of data structures than has been necessary here. We will save a full description of these for a later chapter.

DROP/SHIP

The player deploys from a dropship. This ship is composed of 6 sprites tiled in two rows of three sprites each.

Individually these tiles are unremarkable - - and in the order that they appear it may not even be obvious that together they can form a whole. But when properly combined they give us a chrome dropship from which the player's craft can emerge:

As with the manta craft itself, each of these sprites is built up from 63 bytes of data. On the following page we can see the familiar diagrams from the rear of the dropship in detail.

URIDIUM/DROPSHIP/00

SPEC:DROPSHIP_0 OFFSET:$00 RAM:$4000
AZIMUTH:000° ELEVATION:000° ROLL:000°

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	.BYTE $00,$00,$00
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	0	0	.BYTE $FF,$FF,$F4
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	0	0	1	0	0	0	.BYTE $FF,$FF,$64
1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	0	1	0	1	0	1	0	1	0	0	.BYTE $AA,$A9,$54
1	0	1	0	1	0	1	0	1	0	1	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	.BYTE $AA,$A5,$00
1	0	1	0	1	0	1	0	1	0	0	1	1	0	1	0	1	1	0	1	0	1	0	1	0	0	.BYTE $AA,$9A,$D4
1	0	1	0	1	0	1	0	0	1	1	0	1	1	1	1	0	1	0	1	0	0	0	1			.BYTE $AA,$6F,$51
1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	0	1	0	0	0	0	0	0			.BYTE $FF,$FD,$40
1	0	1	0	1	0	1	0	1	0	1	0	1	0	0	1	0	0	0	0	0	0	0	0			.BYTE $AA,$A9,$00
1	0	1	0	1	1	1	1	1	1	1	0	1	0	0	1	0	0	0	0	0	0	0	0			.BYTE $AF,$E9,$00
1	0	1	1	1	0	1	0	1	0	0	1	1	0	0	1	0	0	0	0	0	0	0	0			.BYTE $BA,$99,$00
1	1	1	0	1	0	1	0	1	0	0	1	1	0	0	1	0	0	0	0	0	0	0	0			.BYTE $EA,$99,$00
1	0	1	0	1	0	1	0	1	0	0	1	1	0	0	1	0	0	0	0	0	0	0	0			.BYTE $AA,$99,$00
0	1	1	0	1	0	1	0	1	0	0	1	1	0	0	1	0	0	0	0	0	0	0	0			.BYTE $6A,$99,$00
0	1	0	1	1	0	1	0	1	0	0	1	1	0	0	1	0	0	0	0	0	0	0	0			.BYTE $5A,$99,$00
1	0	0	1	0	1	0	1	0	1	0	1	1	0	0	1	0	0	0	0	0	0	0	0			.BYTE $95,$59,$00
1	0	1	0	0	1	0	1	0	1	1	0	1	0	0	1	0	0	0	0	0	0	0	0			.BYTE $A5,$69,$00
1	0	1	0	1	0	1	0	1	0	1	0	1	0	0	1	0	0	0	0	0	0	0	0			.BYTE $AA,$A9,$00
1	0	1	0	1	0	1	0	1	0	1	0	1	0	0	1	0	0	0	0	0	0	0	0			.BYTE $AA,$A9,$00
1	1	1	1	1	1	1	1	1	1	1	1	0	1	0	0	1	0	0	0	0	0	0	0			.BYTE $FF,$E9,$00
1	0	1	0	1	0	1	0	1	0	0	1	1	0	0	1	0	0	0	0	0	0	0	0			.BYTE $AA,$99,$00

BASE_COLOR:□ MULTI_COLOR0:■ MULTI_COLOR1:▒

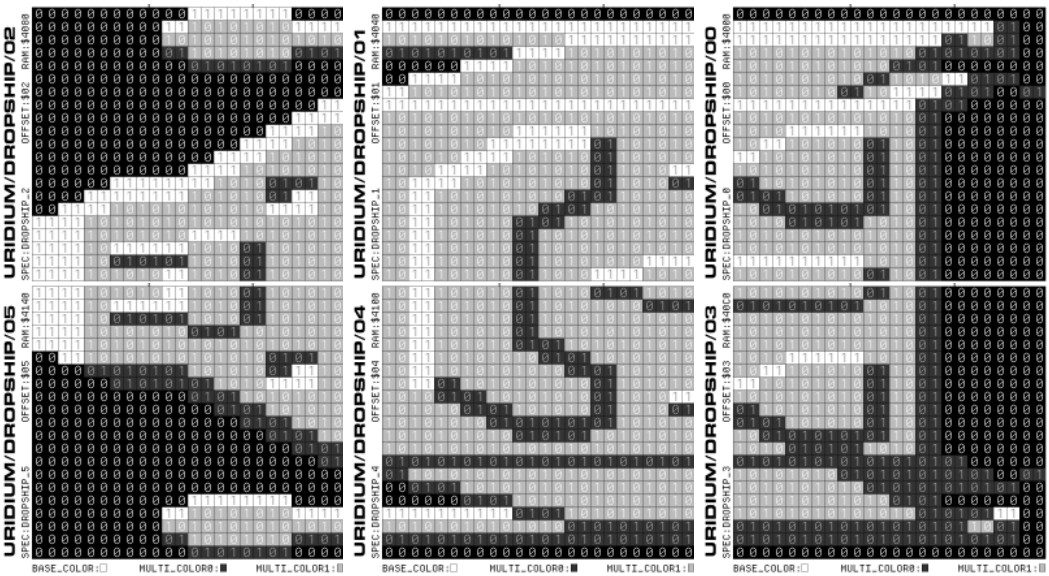
URIDIUM/DROPSHIP/03

SPEC:DROPSHIP_3 OFFSET:$03 RAM:$40C0
AZIMUTH:000° ELEVATION:000° ROLL:000°

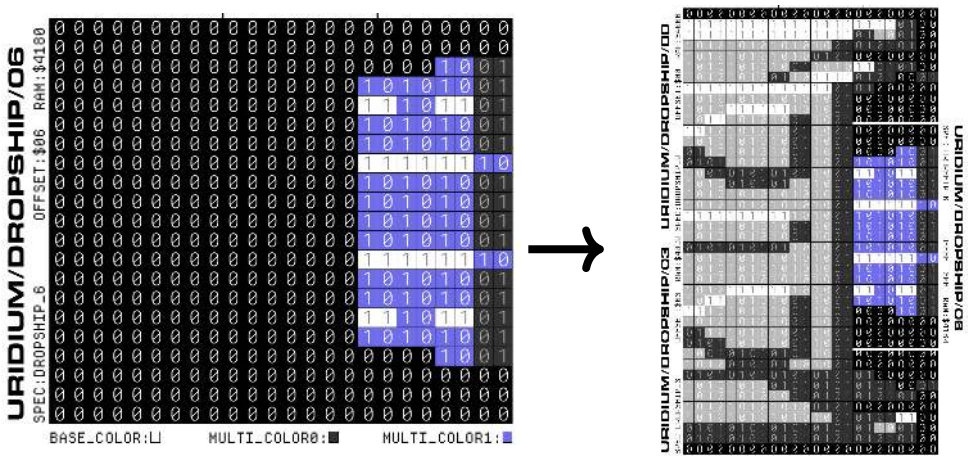
1	0	1	0	1	0	1	0	1	0	0	1	1	0	0	1	0	0	0	0	0	0	0	0			.BYTE $AA,$99,$00
0	1	0	1	0	1	0	1	0	1	1	0	1	0	0	1	0	0	0	0	0	0	0	0			.BYTE $55,$69,$00
1	0	1	0	1	0	1	0	1	0	1	0	1	0	0	1	0	0	0	0	0	0	0	0			.BYTE $AA,$A9,$00
1	0	1	0	1	0	1	0	1	0	1	0	1	0	0	1	0	0	0	0	0	0	0	0			.BYTE $AA,$A9,$00
1	0	1	0	1	0	1	0	1	0	1	0	1	0	0	1	0	0	0	0	0	0	0	0			.BYTE $AA,$A9,$00
1	0	1	0	1	1	1	1	1	1	1	0	1	0	0	1	0	0	0	0	0	0	0	0			.BYTE $AF,$E9,$00
1	0	1	1	1	0	1	0	1	0	0	1	1	0	0	1	0	0	0	0	0	0	0	0			.BYTE $BA,$99,$00
1	1	1	0	1	0	1	0	1	0	0	1	1	0	0	1	0	0	0	0	0	0	0	0			.BYTE $EA,$99,$00
1	0	1	0	1	0	1	0	1	0	0	1	1	0	0	1	0	0	0	0	0	0	0	0			.BYTE $AA,$99,$00
0	1	1	0	1	0	1	0	1	0	0	1	1	0	0	1	0	0	0	0	0	0	0	0			.BYTE $6A,$99,$00
0	1	0	1	1	0	1	0	1	0	0	1	1	0	0	1	0	0	0	0	0	0	0	0			.BYTE $5A,$99,$00
1	0	0	1	0	1	0	1	0	1	0	1	1	0	0	1	0	0	0	0	0	0	0	0			.BYTE $95,$59,$00
1	0	1	0	0	1	0	1	0	1	1	0	1	0	0	1	0	0	0	0	0	0	0	0			.BYTE $A5,$69,$00
0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	0	0	0	0	0			.BYTE $55,$55,$40
1	0	1	0	1	0	1	0	0	1	0	1	0	1	0	1	0	1	0	1	0	0	0	1			.BYTE $AA,$55,$51
1	0	1	0	1	0	1	0	1	0	0	1	0	1	0	1	0	1	0	1	0	1	0	0			.BYTE $AA,$95,$54
1	0	1	0	1	0	1	0	1	0	1	0	0	1	0	1	0	0	0	0	0	0	0	0			.BYTE $AA,$A5,$00
1	0	1	0	1	0	1	0	1	0	1	0	1	0	0	1	0	1	0	1	1	1	0	0			.BYTE $AA,$A9,$5C
0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	1	0	0	1	0	0			.BYTE $55,$55,$64
0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	0			.BYTE $55,$55,$54
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			.BYTE $00,$00,$00

BASE_COLOR:□ MULTI_COLOR0:■ MULTI_COLOR1:▒

But if we zoom in slightly we can also see more about the order in which the sprites are composed.

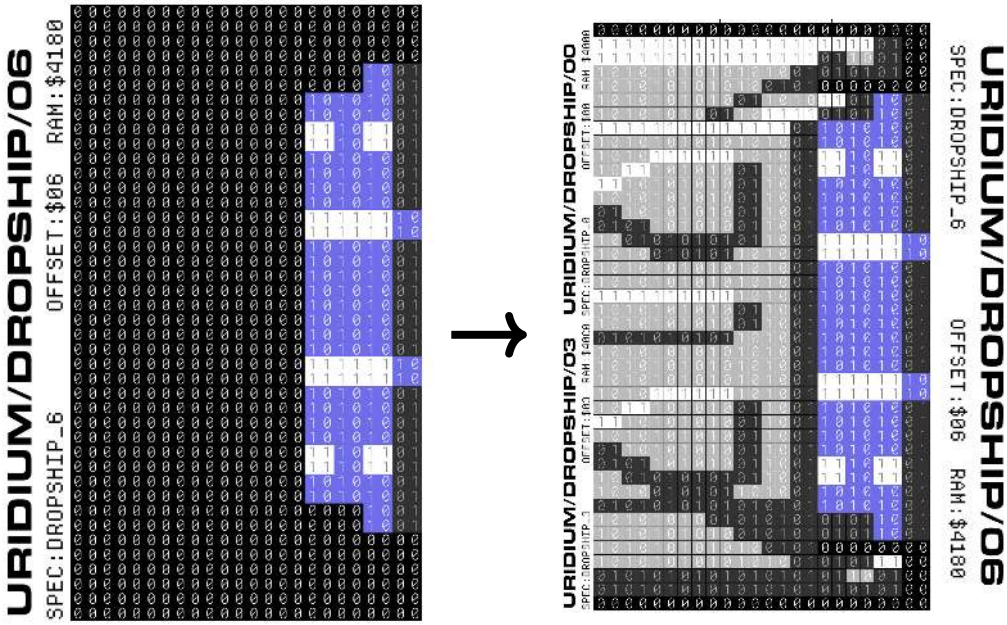


We haven't included the landing bay at the rear of the craft in the montage above. This is because there's a wrinkle we have to take account of in our addition of the sprite that depicts the landing deck. This is evident when we see what happens we attempt it add it to the stern of the dropship below: it's too small!



Why is this? Why doesn't it work like all the other sprites that make up the dropship?

And what do we have to do to make it work? The answer is that we have to render the sprite in a slightly different way. We have to render each row of bits twice. So our rendered sprite will be twice as high as a normal sprite, with 42 rows of pixels rather than 21, and with each of the 21 rows in our data duplicated. How this works in practice is illustrated below. And now the landing deck sprite fits neatly into place in the rear of the drop ship!



Now that we have an idea of how it has to work, let's briefly look at how the composition of the drop ship is implemented in practice. For each section that makes up the ship we will have an array of bytes detailing its position, its color mode, and most importantly the actual sprite value to use. For example, this is what the section array will look like for the section on the top right.

```
topRightSection
  .BYTE $00       ; Sprite's index.
  .BYTE $82       ; Sprite's X position.
  .BYTE $00       ; Sprite's supplemental X position.
  .BYTE $8D       ; Sprite's Y position.
  .BYTE $FF       ; Sprite Enabled/Disabled.
  .BYTE $00       ; Expand Vertical Off.
  .BYTE $00       ; Display Priority vs background.
  .BYTE $FF       ; Multi-Color Mode On.
  .BYTE $00       ; Expand Horizontal Off.
  .BYTE M_GRAY2   ; Sprite color (MULTI_COLOR_1).
  .BYTE [illegible]       ; Sprite Reference ($00).
```

And this is what the section array looks like for the landing deck. Notice that the byte that controls vertical expansion is different from the value used for the `topRightSection`. It is set to $FF, indicating that each row of the sprite should be duplicated, doubling its vertical height as we require.

```
landingDeckSection
  .BYTE $07      ; Sprite's index.
  .BYTE $82      ; Sprite's X position.
  .BYTE $00      ; Sprite's supplemental X position.
  .BYTE $8E      ; Sprite's Y position.
  .BYTE $FF      ; Sprite Enabled.
  .BYTE $FF      ; Expand Vertical ON.
  .BYTE $00      ; Display Priority vs background.
  .BYTE $FF      ; Multi-Color Mode On.
  .BYTE $00      ; Expand Horizontal Off.
  .BYTE M_LTBLUE ; Sprite color (MULTI_COLOR_1).
  .BYTE [illegible]       ; Sprite Reference ($06).
```

With the details of each section defined in its own section array, we will now define an array that references each section array. A 'list of lists' if you like. [1] Taken together, this array tells us the order in which to place the sections. This order is important because when we draw one sprite after another it will appear 'over it', occluding any overlapping area.

```
loPtrsToDropShipSections
  .BYTE <topRightSection,     <topMiddleSection   ; [illegible]
  .BYTE <topLeftSection,      <bottomRightSection ; [illegible]
  .BYTE <bottomMiddleSection, <bottomLeftSection  ; [illegible]
  .BYTE <landingDeckSection,  <bayDoorSection     ; [illegible]
```

Finally, we draw the drop ship using these data structures:

```
  LDX #$07                      ; Start at the end of the section array.
DrawDropShip
  LDA loPtrsToDropShipSections,X ; Get the X'th section from the array.
  STA sectionDataLoPtr           ; Store it.
  LDA hiPtrsToDropShipSections,X ; Get the X'th section from the array.
  STA sectionDataHiPtr           ; Store it.
  JSR LoadAndDisplaySectionData  ; Load the section and display it.
  DEX                            ; Move to the next section..
  BPL DrawDropShip               ; .. until we reach the first one.
```

In this snippet of code we iterate over the array of sections, starting at the end of the

[1] We actually define and use another as well: `hiPtrsToDropShipSections`. This references the sections in the same order as `loPtrsToDropShipSections`. We combine the entries in these arrays to get the two-byte address in memory where our section, such as `topRightSection`, is located and read its data. This is what you see happening in `DrawDropShip`.

array rather than the beginning, and draw each section. This means that the earlier sections will occlude the later ones. Because the landing deck is at the end of the section array this means that any overlapping parts of the deck will be occluded by the section above it, as we would expect if we were looking at in practice.

The illustration below shows how the craft is built up in practice. We start with the landing deck and proceed clockwise along the whole body of the dropship until it is complete.

a random walk through the source code of two video games from the dawn of the computer age